جـامعة الحسن الثـاني بالدار البيضاء
UNIVERSITÉ HASSAN II DE CASABLANCA

ENSAM
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

TP C#

Un duel au tour par tour dans la console · C# pour Unity · Master Technologies Immersives, Ingénierie Digitale pour les Métiers de la Santé

Abdelhakim EL Ghayoubi

ENSAM Casablanca · 2026–2027

Table des matières

Avant de commencer 1
1 Prise en main 3
2 Le duel 7
3 Pour aller plus loin 19
4 Référence 24

Avant de commencer

Une séance d'environ deux heures, dans la console : pas de Unity, pas de scène 3D. Un programme console lit et écrit du texte dans un terminal, ce qui rend chaque étape visible. Vous commencez par quatre exercices courts, puis vous écrivez un petit jeu : la joueuse, Aya, affronte trois ennemis, un par un, au tour par tour. Les exercices de la partie 3 se font à la maison.

Table 1 – Les quatre parties du TP.

Partie	Vous écrivez	Notions principales
1	Quatre petits programmes : une fiche de personnage, un état de santé, un poison, un soin	Variables, `if`, boucle `for`, méthode, lire une erreur
2	Le duel : une joueuse, trois ennemis, une boucle de jeu	Classes, propriétés, héritage, `List<T>`, `while`, saisie sûre
3	Un boss, un événement, plusieurs parties, un fichier de résultats	`virtual`/`override`, `event`, LINQ, `try`/`catch`
4	Rien : c'est la référence (commandes, erreurs, questions fréquentes)	

Tout ce que vous écrivez ici sert dans Unity : une classe `Player` avec une propriété `Health`, une liste d'ennemis, un événement `Died`. Seul l'affichage change. Dans Unity, `Console.WriteLine` devient `Debug.Log`.

Ce qu'il vous faut

- **Le SDK .NET 10**, installé depuis le site de Microsoft (notes de cours, section 0.8).
- **Un éditeur de code** : Visual Studio Code avec l'extension *C# Dev Kit*, ou Visual Studio.
- **Un terminal** : celui de VS Code (menu **Terminal > New Terminal**), ou Windows Terminal.
- **Les notes de cours**, partie 1 : chaque étape donne la section qui explique la notion.

Vérifier l'installation

Ouvrez un terminal et tapez `dotnet --version`. Une version qui commence par `10` doit s'afficher. Si le terminal répond que `dotnet` n'est pas reconnu, voyez la partie 4.



Figure 1 – `dotnet --version` affiche la version du SDK ; `dotnet --list-sdks` liste les SDK installés.

Où ranger les projets

Créez un dossier court, par exemple `D:\TP-CSharp`, et ouvrez-y le terminal (`cd D:\TP-CSharp`). Chaque projet sera un sous-dossier : `Echauffement` pour la partie 1, `Arena` pour le duel. Évitez les chemins longs dans OneDrive : certains outils y échouent.

Comment lire une étape

Les étapes sont numérotées et se terminent par un *point de contrôle* : ce que vous devez voir quand l'étape fonctionne. Ne passez pas à l'étape suivante avant d'avoir atteint le point de contrôle : une erreur se trouve beaucoup plus vite dans dix lignes nouvelles que dans cent.

Tous les affichages de ce document sont de vraies sorties de programmes, et les captures d'écran viennent d'un vrai terminal. Le code des parties 2 et 3 donne la structure et laisse des `// TODO` : c'est à vous d'écrire ces lignes.

> **Attention**
>
> Les combats utilisent des nombres aléatoires. Pour que les sorties de ce document soient reproductibles, elles ont été obtenues avec `new Random(7)` : une *graine* (seed) fixe donne toujours la même suite de nombres. Avec `new Random()`, vos nombres seront différents à chaque partie, et c'est normal.

1 Prise en main

Cette partie vérifie que tout fonctionne et remet en main les bases de la partie 1 du cours : créer un projet, le lancer, écrire quelques lignes, lire une erreur. Chaque exercice tient dans un seul fichier `Program.cs`.

Étape 1 : créer et lancer un projet

1. Dans le terminal, placez-vous dans votre dossier de travail, puis tapez `dotnet new console -n FirstProgram`. La commande crée le dossier `FirstProgram` avec deux fichiers : `FirstProgram.csproj` (la description du projet) et `Program.cs` (le code).
2. Entrez dans le dossier (`cd FirstProgram`) et tapez `dotnet run`. Le SDK compile le projet, puis le lance.
3. Ouvrez le dossier dans VS Code : `code .` (le point désigne le dossier courant).

Point de contrôle : le terminal affiche `Hello, World!`.

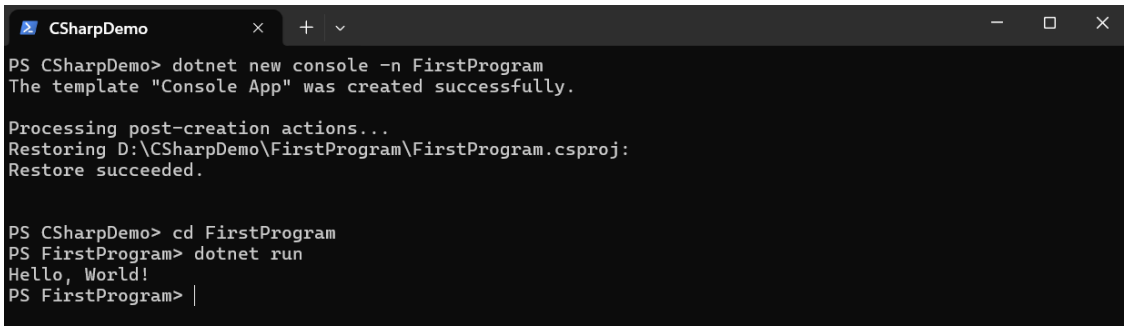


Figure 2 – Les trois commandes et leur résultat. Le premier `dotnet run` prend quelques secondes : il compile avant de lancer.

Pour chaque exercice qui suit, vous pouvez soit créer un nouveau projet (`dotnet new console -n Exercice1`), soit remplacer le contenu de `Program.cs` dans le même projet. Gardez une copie de ce que vous remplacez.

> **Attention**
>
> Le modèle de projet de .NET 10 ajoute lui-même `using System;`. Dans ce TP, chaque fichier l'écrit quand même, comme dans Unity, où il faut l'écrire.

Exercice 1 : une fiche de personnage

Déclarez cinq variables pour Aya : son nom (`string`), son niveau (`int`, 3), sa vie (`int`, 100), sa vitesse (`float`, 5,5) et un booléen qui dit si elle a une clé (`bool`, faux). Affichez-les comme ci-dessous. La dernière ligne est calculée : le niveau suivant demande `niveau × 100` points d'expérience.



Figure 3 – Point de contrôle de l'exercice 1.

Indices. Section 1.2 : un littéral `float` s'écrit avec un `f` (`5.5f`), et le point sert de séparateur décimal dans le code. Une chaîne interpolée, `$"Health: {health} HP"`, remplace chaque `{...}` par sa valeur. Les espaces après `Name:` alignent les valeurs.

> **Attention**
>
> Sur un Windows réglé en français, votre programme affiche `Speed: 5,5 m/s`, avec une virgule. L'affichage d'un nombre suit la langue du système ; le code, lui, s'écrit toujours avec un point. Les projets du cours fixent un affichage neutre, d'où le point dans la capture.

Exercice 2 : l'état de santé

Avec une variable `health`, affichez un message selon la vie d'Aya :

Table 2 – Les quatre états.

Vie	Message
plus de 70	`healthy`
de 31 à 70	`injured`

Vie	Message
de 1 à 30	`critical`
`0`	`Aya is down`

```
Point de contrôle de l'exercice 2, avec health = 45
```

```
Aya has 45 HP: injured
```

Changez ensuite la valeur de `health` (`100`, 12, `0`) et relancez : chaque valeur doit donner son message.

Indices. Section 1.4 : une chaîne `if` / `else if` / `else`. L'ordre des tests compte : C# s'arrête au premier test vrai, donc commencez par le seuil le plus haut.

Exercice 3 : le poison

Aya a `40` points de vie et elle est empoisonnée : à chaque tour, elle perd 8 points. Écrivez une boucle `for` sur six tours qui affiche la vie restante. La vie ne descend jamais sous `0`, et quand elle atteint `0`, le programme affiche `Aya is down.` et sort de la boucle.



Figure 4 – Point de contrôle de l'exercice 3 : la boucle s'arrête au tour 5, avant le sixième.

Indices. Section 1.5 : `for (int turn = 1; turn <= 6; turn++)`. `break` sort de la boucle tout de suite. **Question :** avec un poison de 5 au lieu de 8, que fait le programme ? Prévoyez la réponse, puis vérifiez-la.

Exercice 4 : une méthode de soin

Écrivez une méthode `Heal` qui reçoit la vie actuelle, la quantité de soin et la vie maximale, et qui renvoie la nouvelle vie, sans jamais dépasser le maximum. Appelez-la trois fois :

```
Point de contrôle de l'exercice 4
```

```
50 + 30 -> 80
90 + 30 -> 100
0 + 25  -> 25
```

Indices. Section 1.6 : la signature est `int Heal(int health, int amount, int maxHealth)`, et `Math.Min(a, b)` renvoie le plus petit des deux nombres. Dans un fichier à instructions de haut niveau, la méthode peut être écrite après les lignes qui l'appellent.

Étape 2 : lire une erreur de compilation

Le compilateur refuse un programme qui ne respecte pas les règles du langage, et il dit où et pourquoi. Effacez le `;` à la fin de la ligne 4 de l'exercice 4 (le premier `Console.WriteLine`), puis tapez `dotnet build`. *Point de contrôle :* le build échoue avec ce message.

```
Program.cs(4,53): error CS1002: ; expected
```

Le message se lit de gauche à droite :

1. **Le fichier et la position** : `Program.cs(4,53)` veut dire ligne 4, colonne 53, juste après la parenthèse.
2. **Le code** : `CS1002`. Une recherche « CS1002 » sur Microsoft Learn donne une page qui l'explique.
3. **La phrase** : ce que le compilateur attendait.

Le même oubli ne donne pas toujours le même message. Dans la capture ci-dessous, le `;` manque après `int health = 100`. Une déclaration peut continuer par une virgule (`int a = 1, b = 2;`), donc le compilateur annonce une virgule attendue (`CS1003`) à la fin de la ligne 3. Quand un message semble absurde, regardez ce qui manque à l'endroit indiqué, puis à la fin de la ligne d'avant.

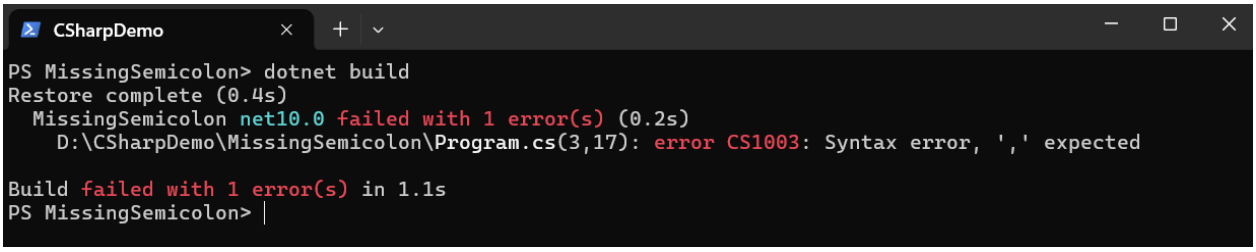


Figure 5 – Un autre point-virgule manquant, dans un autre programme : le message parle d'une virgule.

Remettez le `;`, puis essayez une autre erreur : effacez la dernière `}` du fichier. Le message devient `Program.cs(10,49): error CS1513: } expected`. La partie 4 liste les erreurs que vous rencontrerez le plus souvent dans ce TP (section 1.15 des notes).

2 Le duel

La joueuse, Aya, affronte une vague de trois ennemis, un par un : un Goblin, un Orc, puis un second Goblin. À chaque tour, elle attaque ou boit une potion, puis l'ennemi riposte. La partie se termine quand tous les ennemis sont vaincus, quand Aya n'a plus de vie, ou quand elle s'enfuit.

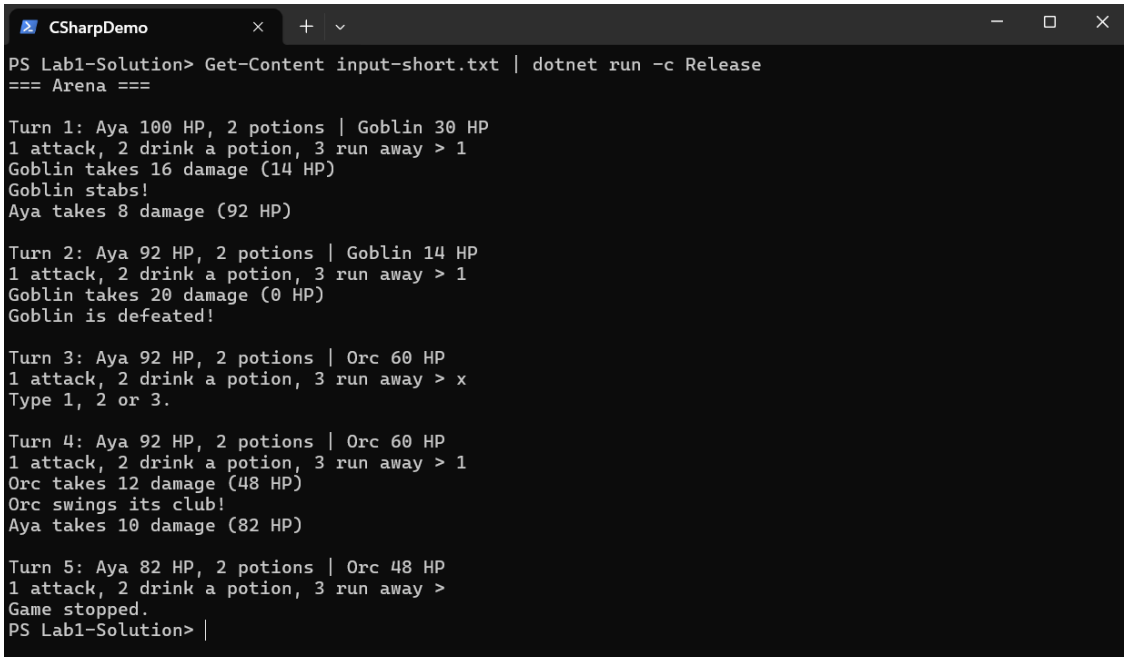


Figure 6 – Le jeu terminé. Ici les choix viennent d'un fichier (1, 1, x, 1) pour que l'exécution soit reproductible ; en séance, vous les tapez au clavier.

Ce que vous allez construire

Le programme a trois sortes d'objets. `Player` garde la vie d'Aya entre `0` et son maximum et compte ses potions. `Enemy` est une classe *abstraite* (abstract) : elle décrit ce que tous les ennemis ont en commun (un nom, une vie, recevoir des dégâts), mais elle laisse à chaque classe fille sa propre façon d'attaquer.

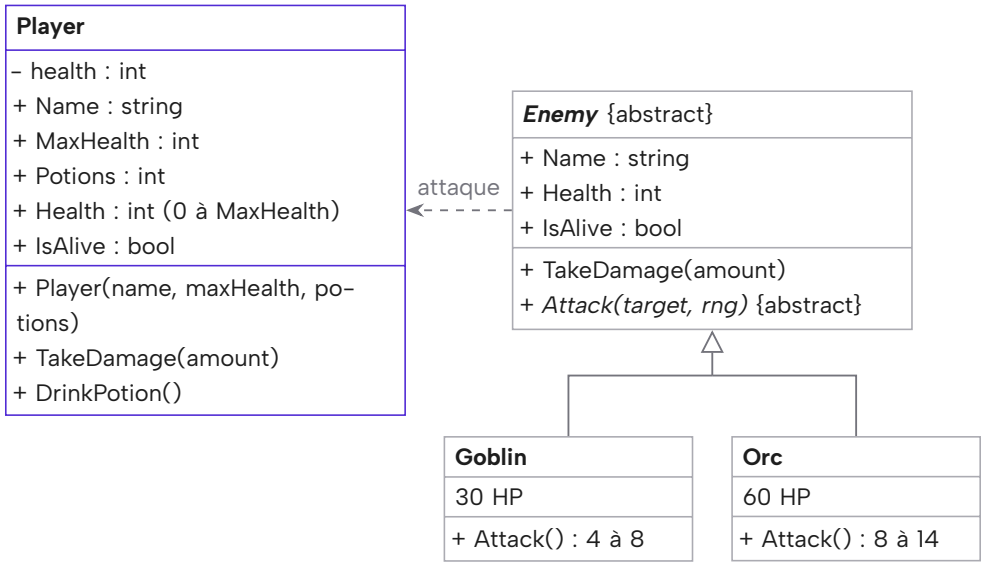


Figure 7 – Les classes du duel. Les nombres sous Attack() sont les dégâts possibles.

Le programme principal crée Aya et la liste des ennemis, puis répète des tours tant qu'Aya est en vie et qu'il reste des ennemis.

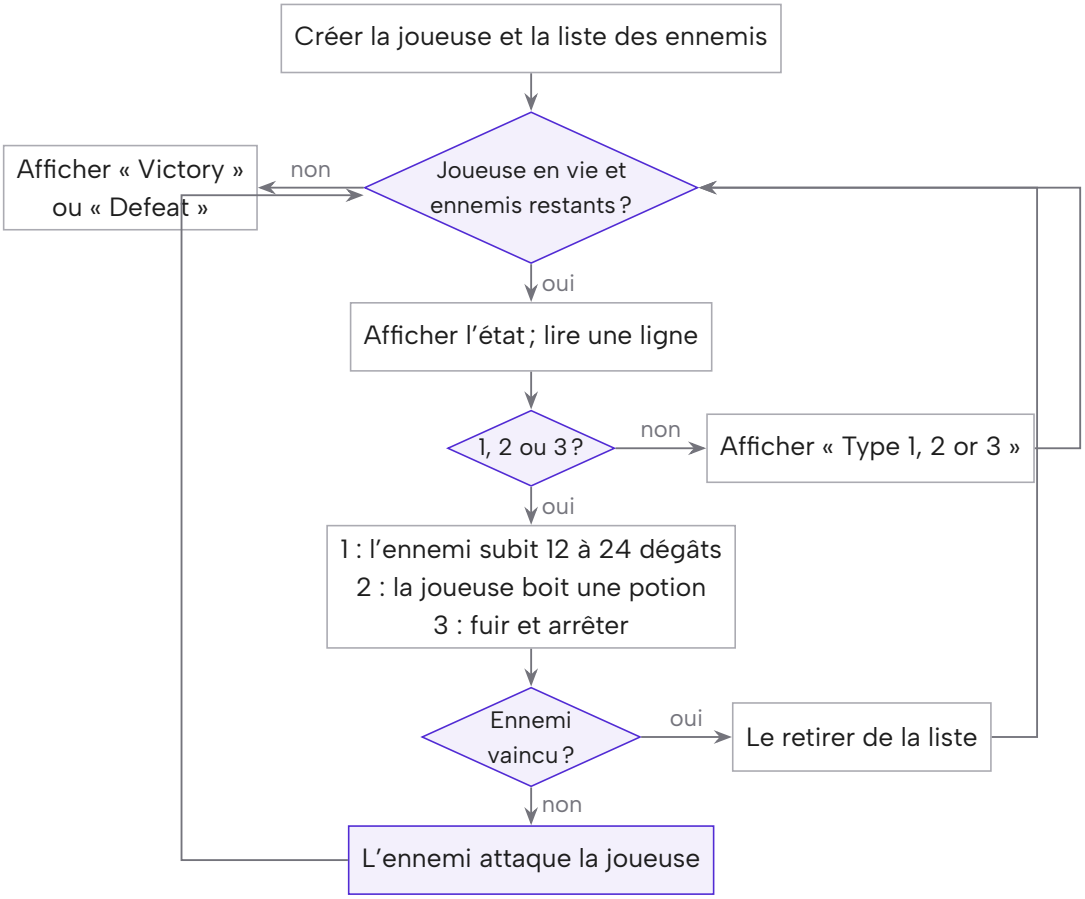


Figure 8 – La boucle de jeu. Chaque passage dans la boucle est un tour. L'étape 5 écrit la boucle sans le choix (Aya attaque toujours) ; l'étape 6 ajoute le choix.

Table 3 – Les sept étapes du duel.

Étape	Vous écrivez	Notion	Notes
1	Le projet `Arena` et le fichier de départ	Projet console	1.1
2	La classe `Player`	Classe, champ privé, propriété, méthode	1.8
3	`Enemy`, `Goblin`, `Orc`	Héritage, méthode abstraite, `Random`	1.10
4	La vague d'ennemis	`List<T>`, `foreach`	1.7, 1.5
5	La boucle de jeu	`while`, `continue`, compteur	1.5
6	Le choix de la joueuse	`Console.ReadLine`, `int.TryParse`, `switch`	1.6, 1.4
7	La fin de partie et les vérifications	Tests à la main	1.15

Étape 1 : le projet

Dans votre dossier de travail : `dotnet new console -n Arena`, puis `cd Arena` et `code .`. Remplacez tout le contenu de `Program.cs` par le fichier de départ ci-dessous (il est aussi distribué avec ce document). Il se compile tel quel ; les `TODO` disent ce que chaque étape ajoute.

`Program.cs (fichier de départ)`

```
// Lab 1 starter: a turn-based duel in the console.
// It builds and runs as it is. Fill in the TODO parts, one step at a time.
using System;
using System.Collections.Generic;

Player player = new Player("Aya", 100, 2);
List<Enemy> enemies = new List<Enemy>();
// TODO step 4: add a Goblin, an Orc and a second Goblin to the list

Console.WriteLine("=== Arena ===");
// TODO step 5: the game loop
//   while the player is alive and enemies remain:
//     show the status, read a choice, act, then the first enemy attacks
// TODO step 7: print "Victory" or "Defeat"

public class Player
{
    // TODO step 2: Name, MaxHealth, Potions, Health (kept between 0 and
    // MaxHealth), IsAlive, TakeDamage(int amount), DrinkPotion()
    public Player(string name, int maxHealth, int potions)
    {
    }
}

public abstract class Enemy
{
    // TODO step 3: Name, Health, IsAlive, TakeDamage(int amount),
    // and an abstract Attack(Player target, Random rng)
}

// TODO step 3: public class Goblin : Enemy { ... }
// TODO step 3: public class Orc : Enemy { ... }
```

Dans ce fichier, les instructions du programme viennent d'abord, les classes ensuite. C# impose cet ordre : une classe placée avant la première instruction donne l'erreur `CS8803` (partie 4).

Point de contrôle : `dotnet run` affiche `=== Arena ===`.



Figure 9 – Point de contrôle de l'étape 1.

Étape 2 : la classe Player

Complétez la classe `Player` en suivant la figure 7. Le champ `health` est privé : seule la propriété `Health` y touche, et son `set` le garde entre `0` et `MaxHealth` avec `Math.Clamp` (section 1.8). Ainsi, ni un gros dégât ni une potion ne peuvent sortir la vie de cet intervalle, quel que soit le code qui appelle `TakeDamage` ou `DrinkPotion`.

`Program.cs : la classe Player, à compléter`

```
public class Player
{
    private int health;

    public string Name { get; }
    public int MaxHealth { get; }
    public int Potions { get; private set; }

    public int Health
    {
        get { return health; }
        private set { health = Math.Clamp(value, 0, MaxHealth); }
    }

    public bool IsAlive => health > 0;

    public Player(string name, int maxHealth, int potions)
    {
        // TODO: copy the three parameters into the properties,
        // then start with full health
    }

    public void TakeDamage(int amount)
    {
        // TODO: lower Health, then print "Aya takes 30 damage (70 HP)"
    }
```

```
    public void DrinkPotion()
    {
        // TODO: no potion left -> print "No potion left!" and stop (return)
        // otherwise: one potion less, 30 health more, print a line
    }
}
```

Pour tester la classe avant d'écrire le jeu, mettez provisoirement ces lignes en haut du fichier, à la place des instructions du départ :

Program.cs : lignes de test de l'étape 2

```
Player player = new Player("Aya", 100, 2);
player.TakeDamage(30);
player.DrinkPotion();
player.DrinkPotion();
player.DrinkPotion();
player.TakeDamage(150);
Console.WriteLine($"Alive: {player.IsAlive}");
```

Point de contrôle : la vie remonte à 100 et pas au-delà, la troisième potion manque, et 150 dégâts laissent la vie à 0, pas à −80.

Sortie de l'étape 2

```
Aya takes 30 damage (70 HP)
Aya drinks a potion (100 HP)
Aya drinks a potion (100 HP)
No potion left!
Aya takes 150 damage (0 HP)
Alive: False
```

Étape 3 : les ennemis

Écrivez la classe abstraite `Enemy` et ses deux classes filles (section 1.10). Le constructeur de `Goblin` appelle celui d'`Enemy` avec `base("Goblin", 30)` : c'est la classe mère qui range le nom et la vie. Chaque classe fille écrit sa propre version de `Attack`, marquée `override`.

Program.cs : les ennemis, à compléter

```
public abstract class Enemy
{
    public string Name { get; }
    public int Health { get; private set; }
    public bool IsAlive => Health > 0;

    protected Enemy(string name, int health)
    {
        Name = name;
        Health = health;
    }

    public void TakeDamage(int amount)
    {
        // TODO: lower Health, never below 0 (Math.Max), print a line
    }

    // no body: each kind of enemy attacks in its own way
    public abstract void Attack(Player target, Random rng);
}

public class Goblin : Enemy
{
    public Goblin() : base("Goblin", 30)
    {
    }

    public override void Attack(Player target, Random rng)
    {
        Console.WriteLine("Goblin stabs!");
        target.TakeDamage(rng.Next(4, 9));     // 4 to 8: the upper bound is excluded
    }
}

// TODO: the Orc, 60 HP, "Orc swings its club!", 8 to 14 damage
```

`Random` produit les nombres aléatoires : `rng.Next(4, 9)` renvoie un entier de 4 à 8, car la borne du haut est exclue. Créez un seul objet `Random` pour tout le programme et passez-le à `Attack` : la graine se fixe alors à un seul endroit, et `new Random(7)` suffit pour rejouer exactement la même partie pendant que vous cherchez une erreur.

Testez avec ces lignes, toujours provisoires :

`Program.cs : lignes de test de l'étape 3`

```
Random rng = new Random(7);          // fixed seed: the same numbers at every run
Player player = new Player("Aya", 100, 2);

Enemy goblin = new Goblin();
Enemy orc = new Orc();
goblin.Attack(player, rng);
orc.Attack(player, rng);
orc.TakeDamage(25);
Console.WriteLine($"{orc.Name} alive: {orc.IsAlive}");
orc.TakeDamage(40);
Console.WriteLine($"{orc.Name} alive: {orc.IsAlive}");
```

Point de contrôle : chaque ennemi attaque avec son propre message, et l'Orc n'est plus en vie après 65 dégâts. Avec `new Random()` au lieu de `new Random(7)`, vos dégâts seront différents.

Sortie de l'étape 3

```
Goblin stabs!
Aya takes 5 damage (95 HP)
Orc swings its club!
Aya takes 14 damage (81 HP)
Orc takes 25 damage (35 HP)
Orc alive: True
Orc takes 40 damage (0 HP)
Orc alive: False
```

Essayez aussi `Enemy e = new Enemy("Bat", 10);` : le compilateur refuse, car une classe abstraite ne se crée pas directement (erreur `CS0144`). Effacez ensuite cette ligne.

Étape 4 : la vague

Remplacez les lignes de test par le vrai début du programme : Aya, puis une `List<Enemy>` remplie avec un Goblin, un Orc et un second Goblin (section 1.7). La liste est de type `Enemy`, mais elle contient des `Goblin` et des `Orc` : un objet d'une classe fille peut se ranger partout où la classe mère est attendue.

Program.cs : début du programme à l'étape 4

```
Player player = new Player("Aya", 100, 2);
List<Enemy> enemies = new List<Enemy>();
enemies.Add(new Goblin());
enemies.Add(new Orc());
```

```
enemies.Add(new Goblin());

Console.WriteLine("=== Arena ===");
Console.WriteLine($"{player.Name} faces {enemies.Count} enemies:");
foreach (Enemy e in enemies)
{
    Console.WriteLine($"- {e.Name}, {e.Health} HP");
}
Console.WriteLine($"First in line: {enemies[0].Name}");
```

Point de contrôle :

Sortie de l'étape 4

```
=== Arena ===
Aya faces 3 enemies:
- Goblin, 30 HP
- Orc, 60 HP
- Goblin, 30 HP
First in line: Goblin
```

Étape 5 : la boucle de jeu

Écrivez la boucle `while` de la figure 8, d'abord sans le choix : à chaque tour, Aya attaque. L'ennemi courant est toujours `enemies[0]`. Quand il est vaincu, `enemies.RemoveAt(0)` le retire et le suivant prend sa place ; `continue` passe au tour suivant, sans riposte d'un ennemi vaincu. Le compteur `turn` augmente de 1 à chaque tour.

Program.cs : la boucle, à compléter

```
Random rng = new Random();
int turn = 0;

Console.WriteLine("=== Arena ===");
while (player.IsAlive && enemies.Count > 0)
{
    turn++;
    Enemy enemy = enemies[0];            // the enemy in front of Aya
    Console.WriteLine();
    Console.WriteLine($"Turn {turn}: {player.Name} {player.Health} HP | {enemy.Name}
    ↪ {enemy.Health} HP");
```

```
    // TODO: Aya attacks: the enemy takes 12 to 24 damage

    // TODO: if the enemy is defeated: print it, remove it from the list
    //       (enemies.RemoveAt(0)) and go to the next turn (continue)

    // TODO: otherwise the enemy attacks Aya
}
// TODO: print "Victory in 8 turns!" or "Defeat..."
```

Point de contrôle : le combat se déroule seul jusqu'au bout et se termine par une victoire ou une défaite.

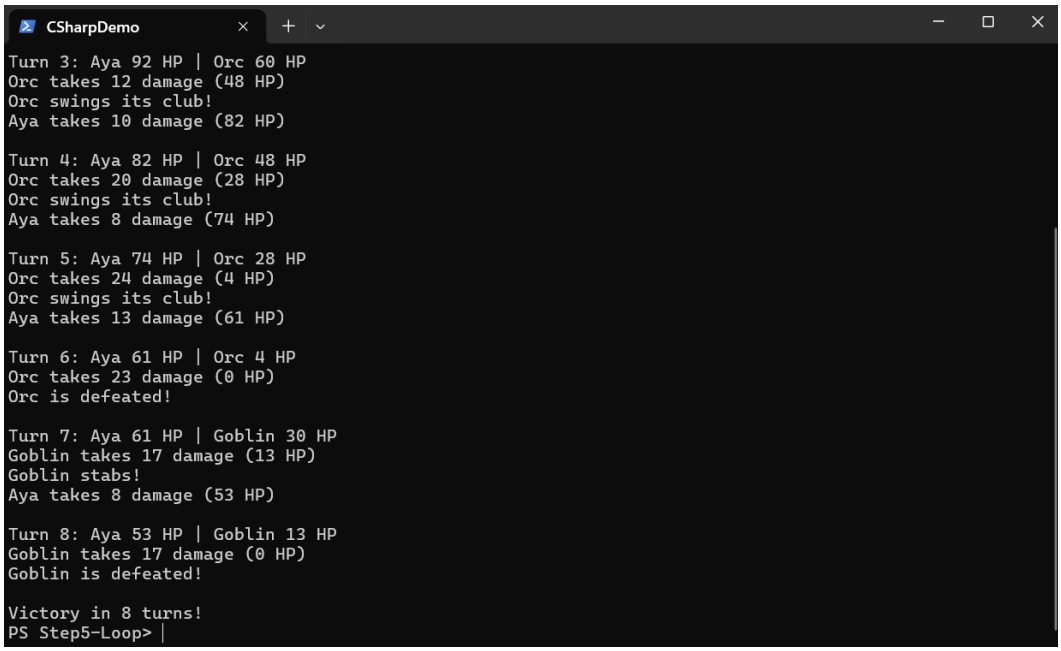


Figure 10 – Point de contrôle de l'étape 5 : la fin d'un combat automatique (graine 7).

> **Attention**
>
> Si le programme ne s'arrête plus et remplit l'écran, la condition du `while` ne devient jamais fausse : un ennemi vaincu n'est pas retiré, ou la vie ne baisse pas. Arrêtez le programme avec **Ctrl+C**.

Étape 6 : le choix de la joueuse

Au début de chaque tour, lisez le choix avec `Console.ReadLine()`. Cette méthode renvoie du texte : pour obtenir un nombre, utilisez `int.TryParse` (section 1.6). Contrairement à `int.Parse`, elle ne lance pas d'exception sur `x` ou sur une ligne vide : elle renvoie `false`. Un `switch` choisit ensuite l'action (section 1.4).

`Program.cs : le choix, dans la boucle, à compléter`

```
Console.Write("1 attack, 2 drink a potion, 3 run away > ");
string text = Console.ReadLine();
if (!int.TryParse(text, out int choice))
{
    Console.WriteLine("Type 1, 2 or 3.");
    continue;                              // same turn again
}
switch (choice)
{
    case 1:
        // TODO: the attack of step 5
        break;
    case 2:
        // TODO: Aya drinks a potion
        break;
    case 3:
        Console.WriteLine("You run away.");
        return;                            // ends the program
    default:
        // TODO: any other number: the same help line, then continue
        break;
}
```

Ce code remplace l'attaque automatique de l'étape 5. Gardez la suite de la boucle : ennemi vaincu ou riposte. Dans le cas 2, la potion prend le tour d'Aya, et l'ennemi riposte quand même. Passez aussi à `new Random()`.

Point de contrôle : une saisie qui n'est pas 1, 2 ou 3 affiche l'aide et redonne le même tour, sans planter.

`Extrait d'une partie : x, puis 1`

```
Turn 3: Aya 92 HP, 2 potions | Orc 60 HP
1 attack, 2 drink a potion, 3 run away > x
Type 1, 2 or 3.

Turn 4: Aya 92 HP, 2 potions | Orc 60 HP
1 attack, 2 drink a potion, 3 run away > 1
Orc takes 12 damage (48 HP)
```

Étape 7 : la fin de partie

Après la boucle, affichez `Victory in 10 turns!` (avec le vrai nombre de tours) ou `Defeat...`. Puis jouez plusieurs parties et vérifiez chaque cas à la main :

- **Victoire** : en attaquant à chaque tour, les trois ennemis tombent.
- **Défaite** : en buvant des potions sans attaquer, Aya finit par perdre ; la vie s'arrête à `0`.
- **Potions** : la troisième potion affiche `No potion left!`.
- **Saisie** : x, `7` et une ligne vide affichent l'aide.
- **Fuite** : `3` affiche `You run away.` et arrête le programme.

Point de contrôle : une partie complète comme celle-ci.

Fin d'une partie complète

```
Turn 7: Aya 61 HP, 2 potions | Orc 4 HP
1 attack, 2 drink a potion, 3 run away > 2
Aya drinks a potion (91 HP)
Orc swings its club!
Aya takes 13 damage (78 HP)

Turn 8: Aya 78 HP, 1 potions | Orc 4 HP
1 attack, 2 drink a potion, 3 run away > 1
Orc takes 17 damage (0 HP)
Orc is defeated!

Turn 9: Aya 78 HP, 1 potions | Goblin 30 HP
1 attack, 2 drink a potion, 3 run away > 1
Goblin takes 24 damage (6 HP)
Goblin stabs!
Aya takes 6 damage (72 HP)

Turn 10: Aya 72 HP, 1 potions | Goblin 6 HP
1 attack, 2 drink a potion, 3 run away > 1
Goblin takes 13 damage (0 HP)
Goblin is defeated!

Victory in 10 turns!
```

Si vous avez fini en avance, passez à la partie 3.

3 Pour aller plus loin

Quatre exercices, à faire à la maison, dans l'ordre. Chacun ajoute au duel une notion de la fin de la partie 1 du cours, et chacun se retrouve tel quel dans Unity. Partez de votre programme de la partie 2. Les sorties ci-dessous viennent d'une version qui fait les quatre exercices ; dans cette version, le choix 3 (fuir) a disparu, car l'exercice 3 propose de rejouer.

Table 4 – Les quatre exercices.

Exercice	Vous ajoutez	Notion	Notes	Dans Unity
1	Un boss avec une armure	`virtual`, `override`, `base`	1.10	Un ennemi qui hérite d'un script commun
2	Un événement `Died`	`event`, abonnement avec `+=`, lambda	1.13	`ScoreCounter` s'abonne à `Collected`
3	Plusieurs parties et les meilleures	Méthode qui renvoie une valeur, LINQ	1.6, 1.14	Trier des scores, chercher des objets
4	Un fichier de résultats	`File.AppendAllText`, `try`/`catch`	1.12	Sauvegarder une partie

Exercice 1 : un boss avec une armure

Ajoutez un `Boss` à la fin de la vague : 60 points de vie et une armure de 6, qui retire 6 points à chaque coup reçu (au moins 1 dégât passe toujours). Le boss doit donc *changer* la façon dont un ennemi reçoit des dégâts.

1. Dans `Enemy`, marquez `TakeDamage` comme `virtual` : une classe fille a maintenant le droit de la remplacer.
2. Dans `Boss`, écrivez `public override void TakeDamage(int amount)`. Elle réduit les dégâts, puis appelle `base.TakeDamage(reduced)` : la version d'`Enemy` fait le reste (baisser la vie, afficher la ligne).
3. Ajoutez `new Boss()` à la liste.

`Program.cs : le boss, à compléter`

```
// in Enemy: virtual allows a child class to replace the method
public virtual void TakeDamage(int amount) { /* as before */ }

public class Boss : Enemy
{
    public int Armour { get; } = 6;

    public Boss() : base("Boss", 60)
    {
    }

    public override void TakeDamage(int amount)
    {
        int reduced = Math.Max(amount - Armour, 1);
        Console.WriteLine($"The armour absorbs {amount - reduced} damage.");
        base.TakeDamage(reduced);          // the version of Enemy does the rest
    }

    // TODO: Attack, "Boss slams the ground!", 9 to 16 damage
}
```

Point de contrôle : chaque coup sur le boss affiche ce que l'armure absorbe.

Un tour contre le boss : 17 dégâts, dont 6 absorbés

```
Turn 7: Aya 61 HP, 2 potions | Boss 60 HP
1 attack, 2 drink a potion > 1
The armour absorbs 6 damage.
Boss takes 11 damage (49 HP)
Boss slams the ground!
Aya takes 16 damage (45 HP)
```

Question. Sans le mot `virtual` dans `Enemy`, que se passe-t-il ? Essayez, lisez l'erreur, puis remettez-le.

Exercice 2 : un événement Died

Aujourd'hui, c'est la boucle de jeu qui vérifie si Aya est en vie. Avec un *événement* (event), c'est `Player` qui annonce sa mort, et n'importe quelle autre partie du programme peut l'écouter, sans que `Player` sache qui écoute (section 1.13). C'est ainsi que les scripts Unity se parlent sans se connaître.

Program.cs : l'événement Died

```
// in Player
public event Action<string> Died;

public void TakeDamage(int amount)
{
    bool wasAlive = IsAlive;
    Health -= amount;
    Console.WriteLine($"{Name} takes {amount} damage ({Health} HP)");
    if (wasAlive && !IsAlive)
    {
        Died?.Invoke(Name);               // ?. : only if someone subscribed
    }
}

// in the game, after creating the player
player.Died += name => Console.WriteLine($"Game over for {name}.");
```

`Action<string>` est le type d'une méthode qui reçoit une `string` et ne renvoie rien. `Died?.Invoke(Name)` appelle tous les abonnés, et le `?.` évite une erreur quand personne n'est abonné. La variable `wasAlive` garantit que l'événement ne part qu'une fois, au moment exact où la vie passe à `0`.

Point de contrôle : la ligne `Game over for Aya.` s'affiche juste après le coup fatal.

Le dernier tour d'une défaite

```
Turn 10: Aya 9 HP, 2 potions | Boss 24 HP
1 attack, 2 drink a potion > 1
The armour absorbs 6 damage.
Boss takes 15 damage (9 HP)
Boss slams the ground!
Aya takes 10 damage (0 HP)
Game over for Aya.
```

Exercice 3 : plusieurs parties et les meilleures

Déplacez la boucle de jeu dans une méthode `int PlayGame()`, qui crée Aya et les ennemis, joue une partie et renvoie le nombre de tours d'une victoire, ou `0` pour une défaite. Le programme principal enchaîne les parties tant que la réponse à `Play again? (y/n)` est `y`, et garde les tours des victoires dans une `List<int>`. À la fin, LINQ trie la liste et garde les trois meilleures : `OrderBy(t => t)` trie du plus petit au plus grand, `Take(3)` garde les trois premiers (section 1.14).

Program.cs : plusieurs parties

```
using System.Linq;

List<int> wins = new List<int>();          // turns of each won game
int game = 0;
while (true)
{
    game++;
    int turns = PlayGame();                // your game loop, moved into a method:
    if (turns > 0)                         // it returns the turns of a victory, or 0
    {
        wins.Add(turns);
    }
    Console.Write("Play again? (y/n) > ");
    if (Console.ReadLine() != "y")
    {
        break;
    }
}
Console.WriteLine($"Victories: {wins.Count} of {game}");
foreach (int t in wins.OrderBy(t => t).Take(3))
{
    Console.WriteLine($"- {t} turns");
}
```

Point de contrôle : après la dernière partie, le bilan et les meilleures victoires.

```
Fin de la troisième partie et bilan
```

```
Saved: Game 3: defeat
Play again? (y/n) > n

Victories: 2 of 3
- 11 turns
- 13 turns
```

Exercice 4 : un fichier de résultats

Après chaque partie, ajoutez une ligne au fichier `results.txt`. `File.AppendAllText` crée le fichier s'il n'existe pas et écrit à la fin sinon. Écrire dans un fichier peut échouer pour des raisons qui ne dépendent pas du programme (fichier verrouillé par un autre programme, disque plein, fichier en lecture seule) : l'appel se met dans un `try`, et un `catch` affiche un message au lieu de laisser le programme planter (section 1.12). Il en faut deux ici, car ces problèmes arrivent sous deux types d'exception : `IOException`

pour un fichier verrouillé ou un disque plein, `UnauthorizedAccessException` pour un fichier ou un dossier où l'on n'a pas le droit d'écrire.

`Program.cs : enregistrer un résultat`

```
using System.IO;

void SaveResult(string line)
{
    try
    {
        File.AppendAllText("results.txt", line + Environment.NewLine);
        Console.WriteLine($"Saved: {line}");
    }
    catch (IOException e)                // file locked by another program, disk full...
    {
        Console.WriteLine($"Could not save the result: {e.Message}");
    }
    catch (UnauthorizedAccessException e) // read-only file, protected folder
    {
        Console.WriteLine($"Could not save the result: {e.Message}");
    }
}
```

Point de contrôle : le fichier `results.txt` apparaît dans le dossier du projet, une ligne par partie.

`results.txt après trois parties`

```
Game 1: victory in 11 turns
Game 2: victory in 13 turns
Game 3: defeat
```

Pour voir un `catch` travailler, passez `results.txt` en lecture seule (clic droit, **Propriétés**, case **Lecture seule**), puis jouez une partie : le programme affiche `Could not save the result: Access to the path '...' is denied.` et continue. Décochez ensuite la case.

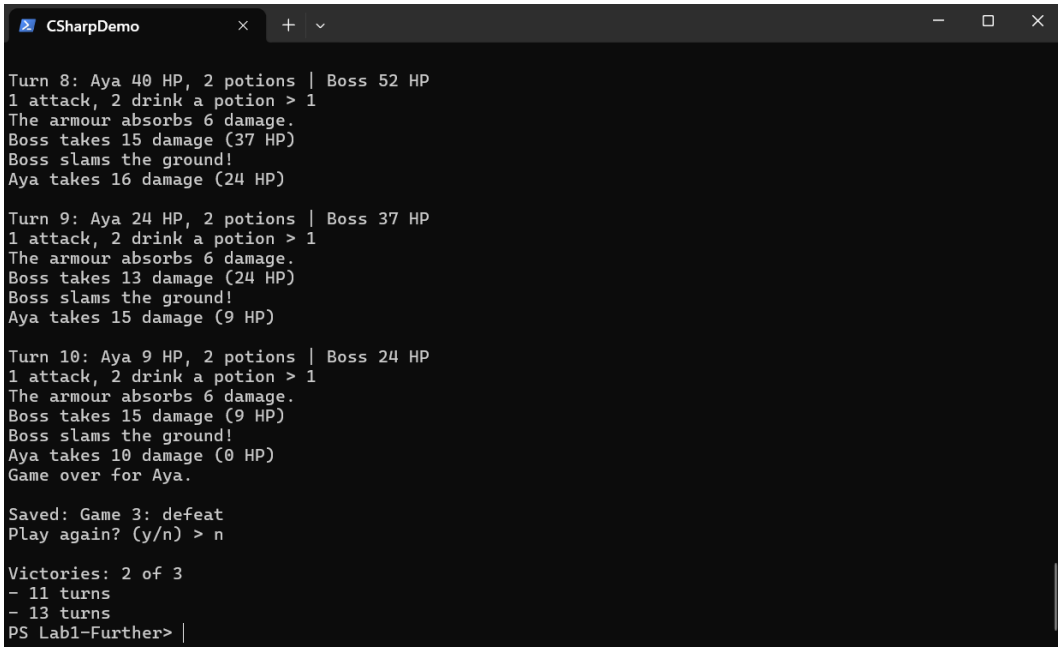


Figure 11 – Les quatre exercices ensemble : la fin d'une défaite, l'enregistrement et le bilan.

4 Référence

Commandes

Table 5 – Les commandes du TP.

Commande	Effet
`dotnet --version`	Affiche la version du SDK installé
`dotnet new console -n Arena`	Crée le dossier `Arena` avec un projet console
`dotnet run`	Compile le projet du dossier courant, puis le lance
`dotnet build`	Compile seulement, et affiche les erreurs
`code .`	Ouvre le dossier courant dans VS Code
Ctrl+C	Arrête le programme en cours dans le terminal
Flèche haut	Rappelle la commande précédente

Messages d'erreur

Les messages ci-dessous sont recopiés de vrais builds. La position `(ligne,colonne)` change selon votre fichier. La section 1.15 des notes explique comment lire un message et en donne d'autres.

Table 6 – Erreurs de compilation fréquentes dans ce TP.

Code	Message	Cause probable	Correction
CS1002	`; expected`	Un `;` manque en fin d'instruction	Ajouter le `;` à la position indiquée
CS1003	`Syntax error, ',' expected`	Un `;` manque après une déclaration	Regarder la fin de la ligne indiquée
CS1513	`} expected`	Une accolade fermante manque	Compter les `{` et les `}` du bloc
CS0103	`The name 'console' does not exist in the current context`	Faute de frappe ou de majuscule, ou variable déclarée ailleurs	`Console`, avec une majuscule ; déclarer la variable avant
CS0246	`The type or namespace name 'List<>' could not be found (are you missing a using directive or an assembly reference?)`	Un `using` manque	`using System.Collections.Generic;`
CS0029	`Cannot implicitly convert type 'string' to 'int'`	Du texte rangé dans un nombre	`int.TryParse` (étape 6)
CS0165	`Use of unassigned local variable 'lives'`	Variable lue avant d'avoir reçu une valeur	Lui donner une valeur à la déclaration
CS0122	`'Player.health' is inaccessible due to its protection level`	Un membre `private` utilisé hors de sa classe	Passer par la propriété `Health`
CS0144	`Cannot create an instance of the abstract type or interface 'Enemy'`	`new` sur une classe abstraite	Créer un `Goblin` ou un `Orc`

Code	Message	Cause probable	Correction
CS0534	`'Orc' does not implement inherited abstract member 'Enemy.Attack()'`	La classe fille n'a pas écrit la méthode abstraite	Écrire `public override void Attack(...)`
CS0506	`'Boss.TakeDamage(int)': cannot override inherited member 'Enemy.TakeDamage(int)' because it is not marked virtual, abstract, or override`	`override` sans `virtual` dans la classe mère	Ajouter `virtual` dans `Enemy`
CS0161	`'Status(int)': not all code paths return a value`	Un chemin de la méthode se termine sans `return`	Ajouter un `return` ou un `else` final
CS8803	`Top-level statements must precede namespace and type declarations.`	Une classe placée avant les instructions	Mettre les classes à la fin du fichier

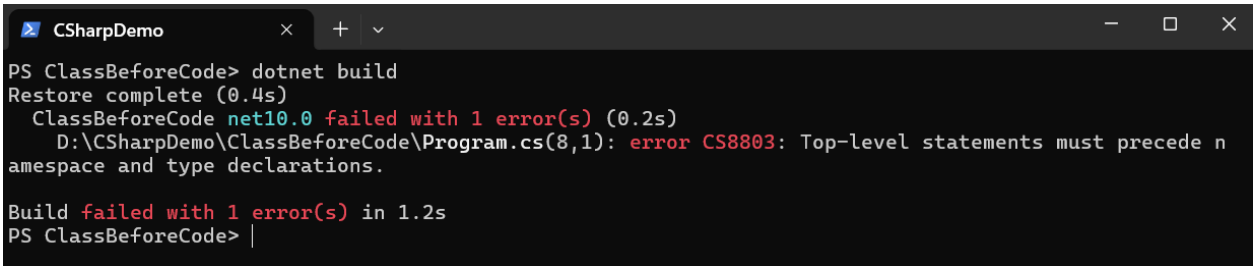


Figure 12 – L'erreur CS8803 : la classe `Player` est écrite avant la première instruction du programme.

Deux erreurs n'apparaissent qu'à l'exécution. Le programme compile, puis s'arrête avec `Unhandled exception` :

Table 7 – Erreurs d'exécution fréquentes dans ce TP.

Exception	Message	Cause probable
`ArgumentOutOfRangeException`	`Index was out of range. Must be non-negative and less than the size of the collection. (Parameter 'index')`	`enemies[0]` sur une liste vide : la boucle ne vérifie pas `enemies.Count > 0`
`FormatException`	`The input string 'x' was not in a correct format.`	`int.Parse` sur une saisie qui n'est pas un nombre : utiliser `int.TryParse`

Questions fréquentes

Le terminal répond que `dotnet` n'est pas reconnu. Le SDK n'est pas installé, ou le terminal a été ouvert avant l'installation. Fermez tous les terminaux (et VS Code), rouvrez-en un, puis retapez `dotnet --version`. Si le problème reste, réinstallez le SDK .NET 10 (notes, section 0.8).

Le programme ne lit pas ce que je tape. Lancez-le avec `dotnet run` dans le terminal. Avec **F5** dans VS Code, selon la configuration, le programme peut tourner dans la *Debug Console*, qui n'accepte pas de saisie.

Le programme ne s'arrête plus. La condition d'une boucle ne devient jamais fausse. **Ctrl+C** l'arrête ; cherchez ensuite ce qui devrait changer à chaque tour (étape 5).

Mes nombres ne sont pas ceux du document. C'est normal avec `new Random()`. Pour obtenir exactement les mêmes, utilisez `new Random(7)`.

La virgule s'affiche à la place du point (`5,5`). L'affichage des nombres suit la langue de Windows. Le code, lui, s'écrit toujours `5.5f`.

VS Code souligne du code en rouge, mais `dotnet run` fonctionne. L'extension n'a pas fini de charger le projet. Attendez quelques secondes, ou fermez et rouvrez le dossier. C'est `dotnet build` qui fait foi.

`Program.cs` contient des erreurs que je ne comprends pas après un copier-coller. Vérifiez qu'il n'y a qu'un seul bloc d'instructions en haut du fichier, et que chaque classe n'apparaît qu'une fois.

Du TP aux notes de cours

Table 8 – Où trouver chaque notion dans les notes de cours.

Notion	Notes	Notion	Notes
Premier programme, `dotnet`	1.1	Classes, propriétés	1.8
Variables et types	1.2	Héritage, `abstract`, `virtual`	1.10
Opérateurs	1.3	Exceptions, `try`/`catch`	1.12
`if`, `switch`	1.4	Délégués et événements	1.13
Boucles	1.5	LINQ	1.14
Méthodes, `int.TryParse`	1.6	Erreurs fréquentes	1.15
`List<T>`	1.7	Du C# à Unity	1.16

Rendu. Compressez le dossier `Arena` sans ses sous-dossiers `bin` et `obj` : `dotnet build` les recrée, et ils sont bien plus gros que le code.